

Full-Stack Coding Assignment – Customer Service Ticketing System

Duration: 3–4 hours

Objective: Build a simplified customer service ticketing system with a **NestJS backend**, **PostgreSQL (via MikroORM)**, and a **ReactJS frontend**.

Problem Statement

Design and implement a customer service ticketing system that allows customers to submit support requests and support agents to manage those tickets.

Ticket Lifecycle

Ticket Statuses

- **OPEN** – Ticket is created and awaiting action
- **IN_PROGRESS** – Ticket is being worked on
- **RESOLVED** – Issue has been resolved
- **CLOSED** – Ticket is finalized (auto-closed)

Allowed Transitions:

OPEN → IN_PROGRESS → RESOLVED → CLOSED

Core Business Rules

1. Tickets are created with status OPEN
2. Tickets cannot skip statuses (e.g., OPEN → RESOLVED is not allowed)

3. Tickets in RESOLVED status are automatically moved to CLOSED after **X days** (default: 3 days)
4. Auto-closing must be handled by a **cron job**
5. Cron job must be idempotent (safe to run multiple times)

A. Backend Requirements (NestJS + MikroORM)

API Endpoints

1. Create Ticket

POST /tickets

Request Body:

```
{
  "customerName": "ABC",
  "customerEmail": "ABC@gmail.com",
  "title": "title",
  "description": "desc",
  ...
}
```

2. List Tickets

GET /tickets?status=OPEN

Optional status filter

3. Update Ticket Status

PUT /tickets/:id/status

Request Body:

```
{  
  "status": "IN_PROGRESS"  
}
```

4. Get Ticket Details

GET /tickets/:id

```
{  
  "status": "OPEN",  
  ...  
}
```

Cron Job Requirement

- Runs once daily (e.g., 02:00 AM)
- Finds tickets with status RESOLVED
- If resolvedAt + X days < currentDate, update status to CLOSED
- Log number of tickets auto-closed

Backend Technical Requirements

- Framework: **NestJS**
- Language: **TypeScript**
- Database: **PostgreSQL**
- ORM: **MikroORM**
- Scheduler: **@nestjs/schedule**
- Input validation using DTOs
- Proper HTTP status codes and error messages

B. Frontend Requirements (ReactJS)

Functional Requirements

1. **Ticket Creation** – Form to create a new ticket (title + description)
2. **Ticket List** – Display list of tickets, filter tickets by status
3. **Ticket Details** – View ticket information, update ticket status using allowed transitions
4. **UI Feedback** – Show loading, success, and error states

Frontend Technical Requirements

- Framework: **ReactJS**
- Global State: **Zustand**
- UI framework: **Ant Design**
- Functional components with Hooks
- API calls using fetch or axios

C. Data Model

```
Ticket {  
  id: string  
  title: string  
  
  customerName: string;  
  
  customerEmail: string;  
  description: string  
  status: 'OPEN' | 'IN_PROGRESS' | 'RESOLVED' | 'CLOSED'  
  createdAt: Date  
  resolvedAt?: Date  
  updatedAt: Date  
}
```

Configuration Requirements

- Auto-close duration (X days) must be configurable
- Configuration via environment variables, database, or configuration file
- All assumptions must be documented

Submission Guidelines

- **Private** GitHub repository, share access to interviewers during interview.
- Recommended structure:

/backend
/frontend
README.md

- Include setup instructions, `.env.example`, migration/seed instructions

Assessment Criteria

Area	Criteria
Functionality	All features work as specified
Business Logic	Correct status transitions & cron behavior
API Design	Clean, consistent REST APIs
Frontend Integration	Correct API usage & UI behavior
Code Quality	Readability, structure, best practices
Error Handling	Proper validation & messaging
Documentation	Clear explanation & assumptions

Bonus (Optional)

- Ticket priority (LOW / MEDIUM / HIGH)
- Assign tickets to agents
- Search by keyword
- **Test cases (optional)** – candidate may provide unit or integration tests for business logic